

Istio Ambient Demo — Full Dry-Run Sheet

From empty cluster to L7 authorization, exactly as on stage. Repo: github.com/borisakov-rapyd/istio-ambient-demo

PHASE 0 — Reset the Git state (5 min)

The cluster is clean (cleanup.sh done). Now reset the **repo values** to the pre-demo baseline:

```
# apps/checkout/values.yaml -> istio.enabled: false          (waypoint: false)
# apps/payment/values.yaml  -> istio.enabled: false, waypoint.enabled: false,
#                             l7Authz.enabled: false, strictMtls: false
git add -A && git commit -m "reset demo baseline" && git push
kubectl config use-context docker-desktop          # or the EKS context on demo day
```

■ both values files show istio.enabled: false · pushed to main · correct kubectl context

PHASE 1 — Bootstrap: one script installs everything (10–15 min)

```
./bootstrap/install.sh          # ArgoCD + root app; prints admin password
while true; do kubectl -n argocd port-forward svc/argocd-server 8080:80; sleep 1; done &
# UI: http://localhost:8080 (admin / password from script output)
kubectl -n argocd get applications -w          # watch the waves roll in
```

Expected apps: root, gateway-api-crds, istio-base, istiod, istio-cni, ztunnel, prometheus, kiali, checkout, payment.

■ **checkout & payment have NO auto-sync (intentional)** — click Sync on both in the UI for the initial install.

```
kubectl -n istio-system get pods          # istiod, istio-cni-node, ztunnel, kiali Running
istioctl ztunnel-config workload | head   # works, but demo apps NOT listed yet
while true; do kubectl -n istio-system port-forward svc/kiali 20001:20001; sleep 1; done &
```

■ all apps Synced/Healthy · demo pods Running in checkout + payment · Kiali opens on :20001

PHASE 2 — The “before” proofs: plaintext & blind (5 min)

Kiali → Traffic Graph → select checkout + payment → auto-refresh 15 s. Graph is EMPTY, workloads show “Out of mesh”. Keep this tab open.

```
kubectl -n checkout exec deploy/checkout -- \
  wget -qO- http://payment.payment.svc.cluster.local/api/charge          # JSON echo, 200

NODE=$(kubectl -n payment get pod -l app=payment -o jsonpath='{.items[0].spec.nodeName}')
kubectl debug node/$NODE -it --image=nicolaka/netshoot -- \
  tcpdump -i any -A -s 120 'tcp port 8080 or tcp port 15008'          # READABLE HTTP on the wire
```

■ wget returns JSON · tcpdump shows plaintext HTTP · Kiali graph empty — keep tcpdump running

PHASE 3 — The flip: one label in Git → mTLS (5 min)

```
# apps/checkout/values.yaml -> istio.enabled: true
# apps/payment/values.yaml  -> istio.enabled: true
git commit -am "enroll checkout & payment into ambient mesh" && git push
# ArgoCD UI: both apps OutOfSync -> open diff (one label on the Namespace) -> Sync both

kubectl get ns -L istio.io/dataplane-mode          # checkout, payment = ambient
kubectl get pods -n checkout -n payment          # same AGE, RESTARTS 0 <- the money proof
istioctl ztunnel-config workload | grep -E 'checkout|payment'          # listed, protocol HBONE
```

tcpdump terminal: port-8080 plaintext is gone — traffic now rides :15008 as TLS noise. Kiali (1–2 min): TCP edge with a padlock (enable the Security badge).

■ namespaces labeled · zero pod restarts · HBONE in ztunnel list · tcpdump encrypted · Kiali padlock

PHASE 4 — L7: waypoint → HTTP metrics → authorization (7 min)

```
# apps/payment/values.yaml -> istio.waypoint.enabled: true (checkout too, for a nicer graph)
git commit -am "attach waypoint" && git push          # -> Sync in UI
kubectl get gateway -n payment          # waypoint Programmed=True
kubectl get pods -n payment          # waypoint-* pod appears
```

Kiali 1–2 min later: HTTP request rates, response codes, latencies replace the plain TCP edge.

```
# apps/payment/values.yaml -> istio.l7Authz.enabled: true (allows GET /api/* only)
git commit -am "L7 authz: GET only" && git push          # -> Sync in UI
kubectl -n checkout exec deploy/checkout-traffic-gen -- \
  curl -s -o /dev/null -w '%{http_code}\n' http://payment.payment.svc.cluster.local/api/charge          # 200
kubectl -n checkout exec deploy/checkout-traffic-gen -- \
  curl -s -w '\n%{http_code}\n' -X DELETE http://payment.payment.svc.cluster.local/api/charge          # RBAC denied / 403
```

The line for the room: **ztunnel decided WHO may connect; the waypoint decides WHAT they may do.**

■ waypoint Programmed · HTTP metrics in Kiali · GET=200 · DELETE=403

PHASE 5 — Optional extras (if time)

```
# STRICT mTLS: apps/payment/values.yaml -> istio.strictMtls: true ; push; Sync
```

```
kubectrl run rogue -n default --image=curlimages/curl --rm -it --restart=Never -- \
  curl -sv --max-time 3 http://payment.payment.svc.cluster.local/api/charge      # connection reset
# live identity stream (great visual):
kubectrl -n istio-system logs ds/ztunnel -f | grep payment
```

Troubleshooting quick table

Apps not syncing	check repo URL reachable; kubectrl -n argocd get applicationsets
ztunnel list empty after flip	namespace label present? kubectrl get ns -L istio.io/dataplane-mode; istio-cni-node logs
Kiali graph empty	prometheus-server Running? wait 2 scrape cycles; traffic-gen pods running?
Port-forward dies	expected — the while-true loops restart it; keep them in dedicated terminals
Everything is on fire	play the recorded rehearsal video. Nobody will judge you.

Reset for the next run

```
./bootstrap/cleanup.sh                                # guarded: confirms context, refuses prod/sbox
DELETE_CRDS=true ./bootstrap/cleanup.sh               # full wipe incl. istio + gateway-api CRDs
# then PHASE 0 again (values back to false, push)
```

Record the clean run end-to-end — that video is the stage fallback if the venue network fails.